

VNUHCM - UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

CSC14003 – STATISTICAL MACHINE LEARNING



FINAL PROJECT - Xây dựng hệ thống trích xuất tài liệu Tiếng Việt

LỚP 23CNTThuc2

Sinh viên:

23127168 – Cao Trần Bá Đạt
23127120 – Trần Danh Thiện

Giảng viên:

Mr. Ngô Minh Nhựt
Mr. Lê Long Quốc

August 20th, 2026

Contents

1	Tổng quan & bài toán	3
1.1	Bối cảnh & Động lực nghiên cứu	3
1.2	Bài toán	3
1.3	Thách thức nghiên cứu	3
1.4	Mục tiêu của đề án	4
2	Dataset và EDA	4
2.1	Giới thiệu tập dữ liệu nghiên cứu	4
2.2	Phân bố và thống kê mô tả	5
2.3	Phân tích cấu trúc câu hỏi có đáp án và không có đáp án	6
2.4	Tiền xử lý & Chuẩn hóa định dạng	6
3	Phương pháp và kiến trúc hệ thống	6
3.1	Kiến trúc mô hình nền tảng	6
3.2	Tiền xử lý Tokenizer & Cửa sổ trượt (QATokenizerProcessor)	7
3.3	Hàm mất mát & Tối ưu hóa (Loss Formulation & Optimization)	8
3.4	Cơ chế suy luận một tầng (Inference & Post-processing)	9
3.5	Tầng hai: Reranker đặc trưng bề mặt & Ngưỡng từ chối hiệu chuẩn	9
4	Thiết lập và đánh giá thí nghiệm	11
4.1	Môi trường tái lập	11
4.2	Độ đo	11
4.3	Kết quả trên tập test đầy đủ ($n = 2882$)	11
4.4	Hồ sơ lỗi: hệ thống sai ở đâu	12
4.5	Huấn luyện: hội tụ và chọn checkpoint	13
4.6	So sánh một tầng và hai tầng	13
4.7	Ngưỡng từ chối τ : đánh đổi đo trên validation	14
4.8	Độ trễ suy luận	14
4.9	Những hướng đã thử và không hiệu quả	14
4.10	Giới hạn	15
5	Ứng dụng Web Demo	15
5.1	Nguyên tắc thiết kế: một lõi suy luận, hai giao diện	15
5.2	Tab 1 — Hỏi đáp trên đoạn văn của người dùng	16
5.3	Tab 2 — Đánh giá hàng loạt trên câu hỏi test thật	16
5.4	Đường dẫn REST	17
5.5	Ảnh chụp màn hình	17
5.6	Giới hạn của bản demo	17
6	Kết luận và hướng phát triển	17
6.1	Kết luận	17
6.2	Hướng phát triển	18
7	References	18
	Tài liệu tham khảo	18

List of Tables

1	Thống kê độ dài dữ liệu theo số từ	6
2	Môi trường thực nghiệm	11
3	Kết quả mô hình một tầng trên UIT-ViQuAD 2.0 test	12
4	Hồ sơ lỗi của mô hình một tầng trên test ($n = 2882$, các nhóm loại trừ nhau)	12
5	Diễn biến <code>eval_loss</code> theo epoch (validation, cùng cấu hình mục 3.3) . .	13
6	Một tầng (pointer thuần) so với hai tầng (reranker + ngưỡng $\tau = 0.95$) — test, $n = 200$	13
7	Quét ngưỡng τ trên validation ($n = 2872$)	14
8	Hai endpoint của <code>src/api.py</code> (đã kiểm chứng bằng <code>curl</code>)	17

1 Tổng quan & bài toán

1.1 Bối cảnh & Động lực nghiên cứu

Trong kỷ nguyên bùng nổ thông tin và chuyển đổi số, khối lượng dữ liệu văn bản tiếng Việt thuộc các lĩnh vực giáo dục, lịch sử, kỹ thuật và đời sống ngày càng trở nên khổng lồ. Việc tìm kiếm thông tin theo phương pháp truyền thống dựa trên từ khóa (Keyword-based Search) thường trả về toàn bộ tài liệu dài, đòi hỏi người dùng phải tự đọc và tổng hợp để tìm ra câu trả lời chính xác.

Để giải quyết hạn chế này, bài toán Máy đọc hiểu và Hỏi - Đáp tự động (Machine Reading Comprehension & Question Answering - MRC/QA) ra đời nhằm mục tiêu giúp máy tính có khả năng tự đọc hiểu đoạn văn bản ngữ cảnh và tự động trích xuất chính xác câu trả lời ngắn gọn cho câu hỏi của người dùng. Việc phát triển một hệ thống MRC chất lượng cao cho tiếng Việt đóng vai trò nền tảng trong việc xây dựng các trợ lý ảo thông minh, hệ thống tra cứu văn bản tự động và các giải pháp giáo dục số.

1.2 Bài toán

Đề án tập trung giải quyết bài toán Hỏi - Đáp trích xuất trên văn bản tiếng Việt có kèm nhận diện câu hỏi không thể trả lời (Vietnamese Extractive Question Answering with Unanswerable Questions) theo định dạng chuẩn của bộ chuẩn SQuAD 2.0 [3] / ViQuAD 2.0 [1, 2].

- **Input:**

- Một đoạn văn bản ngữ cảnh $\mathbf{C} = (c_1, c_2, \dots, c_m)$ gồm m ký tự/từ.
- Một câu hỏi $\mathbf{Q} = (q_1, q_2, \dots, q_k)$ gồm k ký tự/từ liên quan đến nội dung văn bản.

- **Output:**

- Nếu câu hỏi có câu trả lời trong ngữ cảnh (`is_impossible = False`): Mô hình cần trích xuất một chuỗi con liên tục $\mathbf{A} \subseteq \mathbf{C}$ được xác định bởi cặp chỉ số bắt đầu và kết thúc $[start_idx, end_idx]$ sao cho đoạn văn bản tương ứng chứa nội dung trả lời chính xác nhất.
- Nếu câu hỏi không thể trả lời dựa trên ngữ cảnh (`is_impossible = True`): Mô hình phải nhận biết được sự thiếu hụt thông tin hoặc tính phi lý của câu hỏi so với văn bản và trả về kết quả rỗng ($\mathbf{A} = \emptyset$).

1.3 Thách thức nghiên cứu

- **Phân biệt câu hỏi bất/không có đáp án (`is_impossible`):** Dữ liệu chứa các câu hỏi gây nhiễu được thiết kế tinh vi bằng cách hoán đổi thực thể hoặc mốc thời

gian. Mô hình không thể chỉ dựa vào sự trùng lặp từ khóa giữa câu hỏi và đoạn văn mà phải hiểu sâu ngữ nghĩa logic để tránh trích xuất sai lệch.

- **Độ dài ngữ cảnh không đồng nhất:** Các đoạn văn bản có độ dài biến thiên lớn (từ vài chục từ đến hơn 1.500 từ), vượt qua giới hạn độ dài thông thường của các mô hình Transformer (256 hoặc 512 tokens), đòi hỏi kỹ thuật phân mảnh cửa sổ trượt (Sliding Window) và bảo toàn ngữ cảnh.
- **Đặc trưng ngôn ngữ tiếng Việt:** Tiếng Việt là ngôn ngữ có cấu trúc âm tiết phân tích, một đơn vị ngữ nghĩa thường trải trên nhiều âm tiết (thủ tướng, cộng hòa). Bộ tách từ `PhobertTokenizer` của `vinai/phobert-base` nhúng sẵn bước tách từ ghép ở giai đoạn tiền huấn luyện, nên khi tinh chỉnh ta phải kiểm chứng đầu vào dạng nào (nguyên bản hay đã tách) phát huy đúng cơ chế đó — nếu không, mô hình chỉ gặp toàn ký tự chưa từng thấy lúc tiền huấn luyện.

1.4 Mục tiêu của đề án

- Xây dựng pipeline tiền xử lý và số hóa dữ liệu chuẩn hóa: làm phẳng JSON, xuất Parquet, tách cửa sổ trượt và tự triển khai cơ chế ánh xạ tọa độ ký tự sang token index (character-to-token alignment) cho PhoBERT.
- Tinh chỉnh mô hình Transformer chuyên biệt cho tiếng Việt `vinai/phobert-base` trên UIT-ViQuAD 2.0 và đánh giá đúng bộ chỉ số chuẩn của bài toán có câu hỏi bấy.
- Đánh giá toàn diện theo các chỉ số chuẩn học thuật: Exact Match (EM), F1-Score, HasAns EM/F1 và NoAns Accuracy.
- Phân tích định lượng cấu trúc lỗi của mô hình, từ đó đề xuất và kiểm chứng tầng hậu xử lý thứ hai: reranker trên pool ứng viên kèm ngưỡng từ chối hiệu chuẩn (τ).
- Xây dựng giao diện ứng dụng web trực quan (Streamlit) cho phép người dùng nhập ngữ cảnh thực tế, truy vấn tương tác và tự đo lại kết quả trên test set.

2 Dataset và EDA

2.1 Giới thiệu tập dữ liệu nghiên cứu

Đề án sử dụng bộ dữ liệu chuẩn ViQuAD 2.0 (Vietnamese Question Answering Dataset 2.0) — phần câu hỏi có đáp án do nhóm tác giả UIT xây dựng [1], phần câu hỏi bấy không thể trả lời lấy từ phần mở rộng ViMRC [2], theo cấu trúc tiêu chuẩn của bài toán SQuAD 2.0 [3]. Dữ liệu bao gồm các bài viết đa dạng về chủ đề (nhân vật lịch sử, khoa học, kỹ thuật máy tính, sinh học, giáo dục...) gắn liền với lịch sử, được phân rã thành các cấu trúc phân tầng:

-
- **title:** Tiêu đề của chủ đề bài viết (ví dụ: Phạm Văn Đồng, Ngôn ngữ máy, Thực vật có hoa, Edward I của Anh...).
 - **context:** Đoạn văn bản chứa ngữ cảnh tri thức.
 - **qas:** Danh sách các câu hỏi liên quan đến đoạn văn, trong đó mỗi câu hỏi bao gồm:
 - **id:** Mã định danh duy nhất cho từng mẫu câu hỏi.
 - **question:** Nội dung câu hỏi truy vấn.
 - **is_impossible:** Giá trị Boolean xác định câu hỏi có thể trả lời được hay không (**False** là có đáp án, **True** là câu hỏi không thể trả lời).
 - **answers:** Chứa chuỗi đáp án chính xác (**text**) và vị trí ký tự bắt đầu trong đoạn trích (**answer_start**) đối với câu hỏi có đáp án.
 - **plausible_answers:** Chứa câu trả lời hợp lý nhưng sai về mặt logic/thông tin đối với các câu hỏi bẫy (**is_impossible** = **True**), phục vụ việc phân tích ngữ nghĩa.

2.2 Phân bố và thống kê mô tả

Quá trình kiểm tra chất lượng và phân tích thống kê trên tập dữ liệu ghi nhận các chỉ số tổng quan như sau:

- **Kiểm tra tính toàn vẹn và chất lượng dữ liệu:**
 - **Missing Values:** DataFrame không chứa bất kỳ giá trị Null hay NaN bất thường nào; không tồn tại câu hỏi hoặc đoạn văn ngữ cảnh nào bị trống nội dung chuỗi.
 - **Duplicates:** Đạt tính toàn vẹn 100% về mã định danh câu hỏi (0 trường hợp trùng lặp QA ID); chỉ ghi nhận 3 trường hợp trùng lặp cặp nội dung (**Context** + **Question**).
 - **Quy mô ngữ cảnh duy nhất:** Tổng số đoạn văn ngữ cảnh duy nhất trong tập dữ liệu đạt 4.101 đoạn văn.
- **Phân bố câu hỏi có đáp án vs không có đáp án:**
 - Số câu hỏi có câu trả lời (**is_impossible** = **False**): 19.239 câu hỏi ($\approx 67,6\%$).
 - Số câu hỏi không có câu trả lời (**is_impossible** = **True**): 9.217 câu hỏi ($\approx 32,4\%$).
- **Thống kê kích thước văn bản (tính theo số từ):**

Table 1: Thống kê độ dài dữ liệu theo số từ

Thuộc tính thống kê	Ngữ cảnh (Context)	Câu hỏi (Question)	Câu trả lời (Answer)
Độ dài trung bình (μ)	180.6 từ	14.6 từ	6.7 từ
Độ dài tối đa (max)	1.537 từ	53 từ	150 từ

2.3 Phân tích cấu trúc câu hỏi có đáp án và không có đáp án

- **Tỷ lệ câu hỏi `is_impossible`:** Bộ dữ liệu thiết kế tỷ lệ câu hỏi không thể trả lời chiếm gần 1/3 tổng số mẫu. Các câu hỏi này được tạo ra bằng cách thay đổi một chi tiết nhỏ nhưng then chốt so với đoạn văn (ví dụ: thay đổi tên nhân vật thực hiện hành động, thay đổi niên đại sự kiện, hoặc hoán đổi khái niệm).
- **Thách thức bấy ngữ nghĩa:** Thay vì chỉ tìm kiếm sự tương đồng nông về mặt từ vựng, mô hình buộc phải học cách đối sánh vai trò ngữ nghĩa sâu để nhận biết khi nào một câu hỏi dù chứa toàn bộ từ khóa trong bài nhưng lại hoàn toàn không có câu trả lời.

2.4 Tiền xử lý & Chuẩn hóa định dạng

Để phục vụ quá trình huấn luyện và tối ưu hiệu năng I/O trên phần cứng:

- **Làm phẳng dữ liệu (Flattening):** Dữ liệu dạng cây JSON lồng nhau (`title` → `paragraphs` → `qas`) được trích xuất và làm phẳng thành cấu trúc bảng hai chiều (DataFrame).
- **Lưu trữ hiệu năng cao (Apache Parquet):** Chuyển đổi các tập train, validation và test sang định dạng tệp cột nhị phân `.parquet`. Định dạng này giúp giảm kích thước lưu trữ trên đĩa, tăng tốc độ nạp dữ liệu (data loading) và tương thích hoàn toàn với thư viện `datasets` của Hugging Face.

3 Phương pháp và kiến trúc hệ thống

3.1 Kiến trúc mô hình nền tảng

Đề án chọn và tinh chỉnh một mô hình Encoder duy nhất, **vinai/phobert-base** — biến thể RoBERTa [5] được tiền huấn luyện riêng cho tiếng Việt [4]. Lý do lựa chọn:

- **Kiến trúc** (đọc trực tiếp từ `models/phobert_qa/config.json`): 12 tầng Transformer, 12 Attention Heads, kích thước không gian ẩn $d = 768$, tầng `intermediate` 3072, vị trí tối đa 258 token.

- **Từ điển và bộ tách từ:** `vocab_size = 64.001` (token `<mask>` được thêm mới), bộ tách từ `PhobertTokenizer` có 64.000 ký hiệu. PhoBERT khác các mô hình đa ngữ ở chỗ việc tách từ ghép được thực hiện *ngay trong giai đoạn tiền huấn luyện*, nên một token thường tương ứng một đơn vị nghĩa thay vì một mảnh âm tiết tùy ý.
- **Kích thước thực tế:** 134.409.218 tham số (đã gồm hai head tuyến tính start/end), tương ứng 537.660.792 bytes trọng số FP32. Đây là mức vừa với GPU 4GB khi huấn luyện bằng FP16 và gradient checkpointing — ràng buộc quyết định toàn bộ thiết kế thí nghiệm của đề án.

Cả hai tầng suy luận trong đề án đều dùng chung encoder này:

- **Tầng phân loại Hỏi - Đáp (QA Head):** Mô hình được nạp qua `AutoModelForQuestionAnswering` của thư viện Transformers [11], tích hợp một tầng phân loại tuyến tính (Linear Projection) theo thiết kế head span của BERT [7]: $W \in \mathbb{R}^{2 \times 768}$ phía trên tầng ẩn cuối cùng. Với ma trận biểu diễn $H \in \mathbb{R}^{N \times 768}$ của chuỗi N tokens đầu vào, mô hình tính toán hai vector điểm số (logits):
 - $\mathbf{z}^{\text{start}} = HW_{\text{start}}^T \in \mathbb{R}^N$: Điểm số dự đoán token bắt đầu câu trả lời.
 - $\mathbf{z}^{\text{end}} = HW_{\text{end}}^T \in \mathbb{R}^N$: Điểm số dự đoán token kết thúc câu trả lời.

3.2 Tiền xử lý Tokenizer & Cửa sổ trượt (QATokenizerProcessor)

Toàn bộ việc mã hóa do một lớp `QATokenizerProcessor` đảm nhiệm, với các quyết định thiết kế sau:

- **Không có bước tách từ ngoài:** Văn bản tiếng Việt được đưa nguyên dạng vào `PhobertTokenizer`. Khác với mô tả phổ biến về PhoBERT, đề án *không* chạy công cụ tách từ riêng (như `pyvi`) ở tiền xử lý, vì cơ chế ghép âm tiết đã nằm trong bảng từ lúc tiền huấn luyện.
- **Cơ chế Cửa sổ trượt (Sliding Window):** Thiết lập `max_length = 256` và `doc_stride = 64`. Câu hỏi được mã hóa toàn bộ trước, phần ngữ cảnh chỉ còn ngân sách `max_length - |question| - 4` token (định dạng đầu vào `<s> Question </s></s> Context </s>` gồm 4 token đặc biệt). Ngữ cảnh dài được cắt thành các cửa sổ liên tiếp, mỗi cửa sổ tiến một bước `min(length, doc_stride)` token nên luôn đè phủ lên cửa sổ trước, đảm bảo đáp án nằm ở biên đoạn văn xuất hiện nguyên vẹn trong ít nhất một cửa sổ. Chuỗi sau đó được pad đều về 256 token bằng `<pad>` kèm `attention_mask`.
- **Thuật toán ánh xạ tọa độ Ký tự sang Token (Character-to-Token Alignment):**
 - `PhobertTokenizer` của HuggingFace *không* hỗ trợ `return_offsets_mapping`, nên đề án tự xây lại offset: duyệt tuần tự các token, xóa ký tự nối @@, rồi

dò vị trí xuất hiện của token trên văn bản gốc bằng `text.find(clean, pos)` và cập nhật con trỏ `pos`. Kết quả là danh sách `offset_mapping` lưu `(start_char, end_char)` của từng token; các token đặc biệt và phần câu hỏi được gán `(0, 0)`.

- Vùng ngữ cảnh trong chuỗi được xác định bằng số học thay vì `sequence_ids`: `context_start = |q_ids| + 3`, `context_end = context_start + l - 1` với `l` là độ dài của số.
- Trong vùng đó, `start_positions` là token cuối cùng có điểm bắt đầu ký tự $\leq \text{answer_start}$; `end_positions` là token đầu tiên có điểm kết thúc ký tự $\geq \text{answer_end}$. Cách chọn này tự nhiên dung thứ cho việc một đáp án bị trải trên nhiều subword.
- **Gán nhãn Unanswerable & Clipped Span:** Nếu `is_impossible = True`, hoặc cửa sổ hiện tại không phủ *toàn bộ* đáp án (thiếu cả biên đầu lẫn biên cuối), cả `start_positions` và `end_positions` cùng trở về token `<s>` (`cls_index = 0`). Nhờ vậy mô hình học được hành vi “im lặng” mà không cần head riêng, và các mẫu bị cắt nửa do cửa sổ trượt không bị gán nhầm thành một span sai.

3.3 Hàm mất mát & Tối ưu hóa (Loss Formulation & Optimization)

Mô hình quy bài toán trích xuất câu trả lời về việc tối ưu hóa đồng thời hai bài toán phân loại đa lớp trên toàn bộ chuỗi token:

1. Xác suất dự đoán qua Softmax:

$$P_i^{\text{start}} = \frac{e^{z_i^{\text{start}}}}{\sum_{j=1}^N e^{z_j^{\text{start}}}}, \quad P_k^{\text{end}} = \frac{e^{z_k^{\text{end}}}}{\sum_{j=1}^N e^{z_j^{\text{end}}}}$$

2. Hàm mất mát Cross-Entropy: Với nhãn mục tiêu thực tế y^{start} và y^{end} :

$$\mathcal{L}_{\text{start}} = -\log P_{y^{\text{start}}}^{\text{start}}, \quad \mathcal{L}_{\text{end}} = -\log P_{y^{\text{end}}}^{\text{end}}$$

$$\mathcal{L}_{\text{total}} = \frac{\mathcal{L}_{\text{start}} + \mathcal{L}_{\text{end}}}{2}$$

3. Chiến lược tối ưu hóa:

- Bộ tối ưu **AdamW** [8] với learning rate 3×10^{-5} , `warmup_ratio = 0.06` (tăng dần trong 6% đầu số step rồi linear decay) và `weight_decay = 0.01` để hạn chế overfitting.
- Kích thước lô: `per_device_train_batch_size = 8` kết hợp `gradient_accumulation_steps = 2`, cho kích thước lô hiệu dụng 16 trên một GPU 4GB.

- **Tiết kiệm VRAM:** FP16 (`fp16 = True`) giảm một nửa bộ nhớ lưu trọng số và kích hoạt `gradient_checkpointing = True` [9] — đổi một phần tốc độ lấy bộ nhớ kích hoạt, vốn là điều kiện cần để chạy được mô hình 134.4M tham số trên RTX 3050 Laptop 4GB.
- **Lựa chọn checkpoint:** `save_strategy = epoch`, `metric_for_best_model = loss` và `load_best_model_at_end = True`, nên mô hình xuất ra là weights của epoch có `eval_loss` thấp nhất chứ không mặc định là epoch cuối. Toàn bộ thí nghiệm dùng `seed = 42`.

3.4 Cơ chế suy luận một tầng (Inference & Post-processing)

Giai đoạn suy luận của mô hình một tầng đi theo sát thuật toán hậu xử lý của SQuAD 2.0 [3] và được hiện thực trong `src/evaluate.py`, gồm ba bước:

- **Liệt kê ứng viên theo kiểu N-best:** Trong mỗi cửa sổ, lấy 20 vị trí có `start_logits` cao nhất và 20 vị trí có `end_logits` cao nhất, rồi duyệt tích Cartesian của hai tập này với các ràng buộc $i \leq j$ và $j - i + 1 \leq \text{max_answer_length} = 64$. Điểm của một cặp: $\text{score}(i, j) = z_i^{\text{start}} + z_j^{\text{end}}$. Cặp (i, j) thắng độ trên *tất cả* cửa sổ của cùng một câu hỏi trở thành `best_score`, và câu trả lời là `context[start_char : end_char]` khôi phục từ `offset_mapping`.
- **Điểm Null:** Mỗi cửa sổ cho một điểm “không có đáp án” tại chính token `<s>`: $\text{score}_{\text{null}}^{(w)} = z_0^{\text{start}} + z_0^{\text{end}}$. Giá trị dùng để quyết định là **min** trên mọi cửa sổ, $\text{null}_{\text{min}} = \min_w \text{score}_{\text{null}}^{(w)}$ — logic “chỉ cần một cửa sổ tin rằng có đáp án thì trả lời”.
- **Luật quyết định:**

$$\text{Dự đoán} = \begin{cases} (\text{bỏ trống}) & \text{nếu } \text{null}_{\text{min}} > \text{best_score} \\ \text{span khôi phục từ gốc} & \text{ngược lại} \end{cases}$$

Ở đây `best_score` chính là $\max_{i,j} \text{score}(i, j)$.

Hai đại lượng so với nhau trong luật trên đều là **tổng logit chưa chuẩn hoá**, không phải xác suất, và **không có ngưỡng** τ nào ở tầng này. Hệ quả là hành vi “biết nói không biết” phụ thuộc vào thang điểm thô của head pointer: chỉ cần tổng logit của một span dài thừa cao hơn `null_min` một chút là hệ thống trả lời. Con số đo được trên test (mục 4) cho thấy đây đúng là chỗ yếu nhất: chỉ 31.78% câu bẫy được từ chối đúng.

3.5 Tầng hai: Reranker đặc trưng bề mặt & Ngưỡng từ chối hiệu chuẩn

Vì hạn chế ở mục 3.4 nằm ở *quy tắc quyết định* chứ không chỉ ở thứ hạng ứng viên, hệ thống hoàn chỉnh thêm một tầng chấm lại trên cùng một pool ứng viên.

- **Pool ứng viên:** Thay vì chỉ giữ span tốt nhất, `src/qa_service.py` gom mọi ứng viên hợp lệ của mọi cửa sổ, sắp giảm dần theo $\text{score}(i, j)$, **khử trùng lặp theo cặp tọa độ ký tự** (cs, ce) và cắt còn tối đa $\text{top_k_pool} = 40$. Recall của pool này trên val là 92.8% (đáp án vàng nằm trong pool).
- **Vectơ đặc trưng:** Mỗi ứng viên được biểu diễn bằng 34 đặc trưng bề mặt do `src/reranker_features.py` sinh ra, thuộc sáu nhóm: (i) điểm và thứ hạng của extractor kèm độ lệch tương đối so với ứng viên đầu bảng và z-score trong pool; (ii) hình dạng độ dài — số token, số từ, số ký tự, tỉ lệ so với phân phối độ dài đáp án tập train, cờ “dài vượt p95” và cờ “chỉ một token”; (iii) dạng bề mặt — hư từ tiếng Việt (PARTICLES) ở đầu/cuối span, dấu câu ở biên, hoa chữ cái đầu, chứa số; (iv) mức trùng khớp với câu hỏi — chứa nguyên văn và tỉ lệ trùng token; (v) hình học pool — ứng viên này có bị một ứng viên điểm cao hơn *bao chứa* hay không (dấu hiệu kinh điển của cắt biên), số ứng viên điểm thấp hơn nó bao chứa, sự tồn tại của biến thể biên, chỉ số cửa sổ và log số cửa sổ; (vi) ba hiệu số giữa $\text{score}(i, j)$ và các dạng $\text{null}_{\min}$, $\text{null}_{\max}$, top_score .
- **Ứng viên NULL bắt buộc:** Ngoài k ứng viên thật, một hàng giả `\x00NULL` luôn được thêm vào cuối ma trận với đặc trưng riêng (mang $\text{null}_{\min}$, $\text{null}_{\max}$, $\text{top_score} - \text{null}_{\min}$ ở bốn slot dành riêng, các đặc trưng bề mặt đặt bằng 0). Nhờ vậy bài toán trở thành *phân loại nhị phân* “hàng này có phải đáp án đúng không” và câu hỏi “nên im lặng không” cạnh tranh bình đẳng với các phương án trả lời.
- **Mô hình xếp hạng:** `HistGradientBoostingClassifier` của scikit-learn [12], một cài đặt gradient boosting trên cây hồi quy [10]. Mô hình được huấn luyện trên pool của tập validation bằng chính bộ đặc trưng trên; nhãn của một hàng là 1 nếu span trùng khớp tuyệt đối (sau chuẩn hoá) với đáp án vàng, còn hàng NULL nhận nhãn 1 khi và chỉ khi câu hỏi `is_impossible`.
- **Luật từ chối có hiệu chuẩn:** Với $\hat{p}$ là xác suất lớp 1 do reranker trả về,

$$\text{abstain} \iff (\text{không có ứng viên nào}) \vee (\hat{p}_{\text{null}} > \hat{p}_{\text{best}} \wedge \hat{p}_{\text{null}} \geq \tau), \quad \tau = 0.95$$

Sở dĩ cần *cả hai* điều kiện: xác suất NULL của mô hình tăng cường độ lệch thường bảo hoà gần 1, nên chỉ một so sánh hơn-kém rất dễ “im lặng” quá đà. Quét τ trên val (mục 4.7) cho thấy chỉ dải $[0.88, 1.00]$ là có tác dụng và đỉnh F_1 rơi tại $\tau = 0.95$.

- **Chi phí suy luận:** Điểm reranker *không* đổi theo τ , nên GPU chỉ chạy encoder một lần cho một câu hỏi; việc quay ngưỡng sau đó tức thời. Đây là lý do tab đánh giá hàng loạt trong web app cho phép kéo thanh “Độ khắt khe” mà không phải gọi lại mô hình.

Cần nói thẳng giới hạn của tầng hai: trên 821 câu mà extractor xếp hạng sai, reranker chỉ sửa được 12 câu, tức **không** giải quyết được bài toán ranking — gốc rễ vẫn nằm ở kiến trúc pointer (mục 4.6). Giá trị còn lại — và cũng là giá trị chính — của nó là **abstention có hiệu chuẩn**: biến một so sánh logit thô thành một xác suất có ngưỡng điều khiển được.

4 Thiết lập và đánh giá thí nghiệm

4.1 Môi trường tái lập

Toàn bộ thí nghiệm chạy trên một máy cá nhân duy nhất; bảng dưới liệt kê đúng phiên bản đã được cài đặt lúc sinh ra các số liệu trong báo cáo này.

Table 2: Môi trường thực nghiệm

Thành phần	Giá trị đo được
GPU	NVIDIA GeForce RTX 3050 Laptop, 4096 MiB
Python	3.10.11
PyTorch	2.9.1+cu128
Transformers / Datasets	4.57.1 / 4.2.0
scikit-learn / joblib	1.7.2
numpy / pandas	2.2.6 / 2.3.3
Streamlit	1.63.0
seed	42 (mọi thí nghiệm)

4.2 Độ đo

Độ đo là chuẩn SQuAD, hiện thực trong `src/evaluate.py`. Chức năng chuẩn hoá `normalize_text` gồm ba bước: chuyển chữ thường, **xoá toàn bộ dấu câu**, và nén khoảng trắng thừa. Không có bước bỏ mạo từ (“a/an/the”) như bản SQuAD gốc tiếng Anh vì tiếng Việt không có mạo từ.

- **EM (Exact Match)**: 1 nếu chuỗi đã chuẩn hoá của dự đoán trùng khít chuỗi đã chuẩn hoá của đáp án vàng, ngược lại 0.
- **F1 theo token**: với P là tập token dự đoán và G là tập token vàng,

$$\text{Precision} = \frac{|P \cap G|}{|P|}, \quad \text{Recall} = \frac{|P \cap G|}{|G|}, \quad F_1 = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$$

Nếu cả hai đều rỗng thì $F_1 = 1$; nếu một trong hai rỗng thì $F_1 = 0$.

- **HasAns EM/F1**: tính riêng trên câu có đáp án. **NoAns Accuracy**: tỉ lệ câu bẫy (`is_impossible`) mà hệ thống bỏ trống. Hai độ đo sau bắt buộc phải báo cùng hai độ đo đầu, vì một hệ thống luôn luôn trả lời sẽ đạt $\text{NoAns} = 0$ nhưng HasAns lại bị thổi phồng.

4.3 Kết quả trên tập test đầy đủ (n = 2882)

Số liệu trong bảng là output của `src/evaluate.py` chạy trên `data/processed/test.parquet` (1938 câu có đáp án, 944 câu bẫy) và đã được kiểm chứng lại bằng cách chấm trực tiếp

trên `predictions.json`.

Table 3: Kết quả mô hình một tầng trên UIT-ViQuAD 2.0 test

Độ đo	Giá trị
EM toàn bộ	42.40
F1 toàn bộ	57.90
EM câu có đáp án (HasAns)	47.57
F1 câu có đáp án (HasAns)	70.62
Độ chính xác câu bẫy (NoAns)	31.78
Tỉ lệ hệ thống chịu trả lời	82.3
F1 riêng trên 1729 câu có đáp án mà model chịu trả lời	79.16

Hàng cuối là phát hiện định hướng toàn bộ phần còn lại của dự án: *khi model chịu nói, nó nói rất đúng* ($F_1 \approx 79$). Khoảng cách giữa 79.16 và 70.62 — và giữa 70.62 với 57.90 — không đến từ việc model thiếu kiến thức, mà đến từ việc nó không biết dừng lại.

4.4 Hồ sơ lỗi: hệ thống sai ở đâu

Phân loại 2882 câu theo đúng một trong tám nhóm (bảo toàn tổng), so sánh chuỗi đã chuẩn hoá của dự đoán và đáp án vàng:

Table 4: Hồ sơ lỗi của mô hình một tầng trên test ($n = 2882$, các nhóm loại trừ nhau)

Nhóm lỗi	SL	Tỉ lệ	Giải thích
Đúng (có đáp án)	922	31.99	$EM = 1$
Bịa đáp án cho câu bẫy	644	22.35	trả lời một câu không có đáp án
Cắt dài thừa	375	13.01	span chứa cả đáp án vàng cộng nhiều
Bỏ đúng câu bẫy	300	10.41	từ chối đúng
Cắt ngắn thiếu	233	8.08	span là một phần của đáp án vàng
Từ chối oan	209	7.25	bỏ trống câu thực ra có đáp án
Trùng một phần, sai biên	110	3.82	overlap token nhưng không bao chứa
Sai hoàn toàn	89	3.09	không giao nhau về token
Tổng hai nhóm cắt biên	608	21.10	$375 + 233$

Ba nhóm *in đậm* chiếm 1227 câu (42.6%) và là mục tiêu của tầng hai: $644 + 209 = 853$ câu thuộc bài toán *quyết định có trả lời hay không*, 608 câu thuộc bài toán *chọn đúng biên*.

4.5 Huấn luyện: hội tụ và chọn checkpoint

Table 5: Diễn biến `eval_loss` theo epoch (validation, cùng cấu hình mục 3.3)

Epoch	Step	eval_loss	Ghi chú
1	2207	1.3073	
2	4414	1.2203	checkpoint được chọn
3	6621	1.2667	bắt đầu overfit

Tổng thời gian huấn luyện 3537.8 giây (≈ 59 phút) cho 6621 step. Vì `load_best_model_at_end = True`, trọng số xuất ra ở `models/phobert_qa/` *không phải* epoch cuối mà là epoch 2; điều này đã xác minh bằng `sha256sum: model.safetensors` khớp khít với `checkpoint-4414` và *khác* với `checkpoint-6621`. Đây là chi tiết dễ bị bỏ sót khi đọc `trainer_state` và là lý do mọi số liệu trong báo cáo được tái kiểm tra bằng cách chấm lại chính bộ trọng số đã bàn giao.

4.6 So sánh một tầng và hai tầng

Chăm hai hệ thống trên cùng một pipeline, cùng luật hậu xử lý, chỉ khác nhau tầng reranker, trên mẫu 200 câu test rút ngẫu nhiên (`seed = 42`, `src/batch_eval.py`, $\tau = 0.95$).

Table 6: Một tầng (pointer thuần) so với hai tầng (reranker + ngưỡng $\tau = 0.95$) — test, $n = 200$

Độ đo	Một tầng	Hai tầng	Δ
EM toàn bộ	41.50	46.00	+4.50
F1 toàn bộ	56.69	58.92	+2.23
EM câu có đáp án	44.78	41.04	-3.73
F1 câu có đáp án	67.45	60.33	-7.12
Độ chính xác câu bẫy	34.85	56.06	+21.21
Tỉ lệ bỏ trả lời	22.0	35.5	+13.50

Cột “một tầng” (41.50/56.69) sát với số toàn test (42.40/57.90), chứng tỏ mẫu 200 câu không bị lệch và phép so sánh là công bằng.

Kết quả này cần đọc thẳng thắn: **reranker không làm model chọn span giỏi hơn**. Bằng chứng là chỉ số đo đúng việc chọn span (HasAns F1) *giảm* 7.12 điểm. Cơ chế tăng F_1 toàn bộ hoàn toàn nằm ở quyền từ chối: câu bẫy được bắt đúng tăng +21.21. Đào sâu hơn, trên validation pool 40 ứng viên đạt recall 92.8% — tức đáp án đúng có nằm trong pool ở 93% câu — nhưng reranker đặc trưng bề mặt chỉ sửa được 12 trên 821 câu mà extractor xếp hạng sai. Kết luận kỹ thuật: khi head pointer chấm start và end độc lập, lỗi biên là lỗi của *kiến trúc*, không sửa được bằng hậu xử lý bề mặt.

4.7 Ngưỡng từ chối τ : đánh đổi đo trên validation

Vì τ chỉ ảnh hưởng luật quyết định, có thể quét toàn dải mà không cần chạy lại GPU. Validation, $n = 2872$:

Table 7: Quét ngưỡng τ trên validation ($n = 2872$)

τ	EM	F1	Câu bẫy	F1 có đáp án
0.90	46.41	57.89	58.27	57.70
0.95 (mặc định)	45.33	58.76	50.59	62.72
0.97	43.04	58.07	36.93	68.31
0.99	40.46	56.33	24.33	71.83
Không bao giờ từ chối	33.74	50.52	0.00	74.98

Ba nhận xét từ bảng:

- Đỉnh F_1 nằm tại $\tau = 0.95$, và đó là lý do giá trị này thành mặc định.
- Đường cong **đơn điệu và đối chọi được**: tăng τ luôn đổi câu bẫy lấy F_1 có đáp án. Một hệ thống “không bao giờ từ chối” mất 8.24 điểm F_1 toàn bộ (50.52 so với 58.76) dù F_1 câu có đáp án cao nhất — số này minh hoạ định lượng vì sao quyền từ chối phải là một phần của hệ thống.
- Toàn bộ biến động có ý nghĩa chỉ xảy ra trong dải $[0.88, 1.00]$. Bên dưới 0.88, không phương án nào vượt được τ và hành vi bảo hoà — một dấu hiệu điển hình của xác suất chưa hiệu chuẩn.

4.8 Độ trễ suy luận

Đo bằng `time.perf_counter` quanh một lần gọi `QAService.answer()` trên RTX 3050 (FP16), bao gồm cả encoder và reranker:

- Nạp mô hình một lần: 17–21 giây.
- Câu ngắn (một cửa sổ): ≈ 40 –90 ms.
- Toàn bộ tập test: 190–915 ms/câu tùy độ dài đoạn văn; 60 câu hết 16–22 giây.

4.9 Những hướng đã thử và không hiệu quả

Ba hướng sau đã được triển khai và đo, và **bị loại vì số liệu**, ghi lại ở đây để khoanh vùng phạm vi kết luận:

-
- **Đổi backbone sang xlm-roberta-base [6]:** không được huấn luyện, nên không có số để so sánh. Lý do bỏ: chi phí huấn luyện lại trên GPU 4GB không tương xứng với kỳ vọng cải thiện lỗi biên — thứ thuộc về head pointer chứ không thuộc về encoder.
 - **Trim span từ hai đầu và chỉnh ngưỡng trên logit pointer:** không sửa được nhóm lỗi cắt biên (608 câu).
 - **Multi-task head và sinh dữ liệu ngược:** đã thử ở nhánh trước, không cho tăng trưởng đo được trên validation.

4.10 Giới hạn

- Phép so sánh hai tầng thực hiện trên mẫu 200 câu, không phải toàn bộ 2882 câu, do chi phí thời gian trên GPU 4GB. Mẫu khớp với toàn test ở cột nền nhưng Δ vẫn mang sai số lấy mẫu.
- Mô hình reranker được huấn luyện trên chính tập validation, nên các số ở mục 4.6 và 4.7 *không* hoàn toàn độc lập với dữ liệu huấn luyện của nó. Test set không hề tham gia vào quá trình nào, nhưng khoảng “val để chọn τ ” là hẹp.
- AUC 0.929 của reranker trên tập do chính nó huấn luyện là con số lạc quan quá mức và đã bị loại khỏi báo cáo khi phát hiện là hiện tượng thẩm định vòng lặp.
- Không có đường cơ sở ngoài: toàn bộ so sánh là nội bộ giữa các biến thể của cùng một hệ thống. F_1 test 57.90 nên hiểu là kết quả tự lặp lại được trong môi trường mục 4.1, không phải một xếp hạng trên bảng công bố.

5 Ứng dụng Web Demo

5.1 Nguyên tắc thiết kế: một lõi suy luận, hai giao diện

Toàn bộ logic suy luận nằm trong `QAService` (`src/qa_service.py`, 249 dòng) và **chỉ nằm ở đó**. Encoder, bộ trả lời cửa sổ trượt, pool 40 ứng viên và reranker đều được gọi qua đúng một hàm `answer(question, context, top_k, tau_null)`. Hai tầng giao diện chỉ khác nhau ở lớp vỏ:

- `app/app.py` (468 dòng) — giao diện Streamlit cho người dùng cuối.
- `src/api.py` (109 dòng) — REST API FastAPI hướng dẫn chương trình.

Giao diện web dùng Streamlit [13]. Nhờ dùng chung một lõi, số liệu hiển thị trên web và số liệu `src/batch_eval.py` chấm hàng loạt là *cùng một đường chạy*, không phải hai bản cài đặt song song vốn dễ lệch nhau. Đây là điểm mấu chốt khiến tab đánh giá hàng loạt

đáng tin: những gì người dùng thấy trên giao diện chính là những gì đã được báo cáo ở mục 4.

Mô hình được nạp một lần duy nhất nhờ `@st.cache_resource` của Streamlit (17–21 giây cho trọng số 537 MB); mọi thao tác sau đó trên cùng một phiên đều dùng lại tiến trình đã load.

5.2 Tab 1 — Hỏi đáp trên đoạn văn của người dùng

Người dùng dán một đoạn văn tiếng Việt và một câu hỏi, kèm thanh trượt “*Độ khát khi nói ‘không có đáp án’*” (chỉnh τ , dải 0.90–0.99, mặc định 0.95). Kết quả trả về:

- **Hoặc** câu trả lời trích xuất được, **hoặc** thông báo model từ chối, kèm phương án nó sẽ chọn nếu buộc trả lời và gợi ý hạ τ .
- Bốn chỉ số: độ tin cậy của phương án được chọn (kèm ngưỡng đang áp dụng), số ứng viên đã cân nhắc, số cửa sổ token, và độ trễ đo bằng `time.perf_counter` quanh đúng một lần gọi hàm.
- Đáp án được **tô đậm ngay trong đoạn văn** để kiểm tra tại chỗ rằng nó là văn bản gốc, không phải câu văn do hệ thống dựng lại.
- Bảng **mọi phương án đã so sánh** với độ tin cậy và số token của từng phương án.
- Mục so sánh với mô hình một tầng: model gốc (không reranker) chọn gì, kèm bảng đánh đổi của τ đo trên validation.

5.3 Tab 2 — Đánh giá hàng loạt trên câu hỏi test thật

Tab này là phần khác biệt nhất của ứng dụng: thay vì để người dùng tự thử vài câu, nó chạy *chính câu hỏi trong test set* qua cả hai tầng rồi tự chấm bằng đúng độ đo ở mục 4.2.

- Mẫu 20–300 câu (mặc định 60), `seed = 42` nên chạy lại vẫn ra đúng số cũ; tỉ lệ câu bẫy được giữ nguyên 32.8% như tập đầy đủ.
- GPU chạy **đúng một lần cho mỗi câu**. Vì điểm các phương án không đổi theo τ , thanh trượt τ (0.88–1.00, bước 0.005) tính lại toàn bộ bảng và đường cong trong vài mili giây mà không gọi lại mô hình.
- Năm chỉ số hiển thị dạng Δ so với mô hình một tầng: EM toàn bộ, F1 toàn bộ, F1 câu có đáp án, tỉ lệ bắt câu bẫy, tỉ lệ bỏ trả lời. Hai cột so sánh được vì chúng *chung một lần encode*.
- Đường cong đánh đổi (Plotly) của F1 / câu bẫy / bỏ trả lời theo τ , có vạch đánh dấu ngưỡng đang xem.

- Bảng “từng câu một” với bộ lọc theo kết luận (sửa được / làm hỏng / không đổi), và khung soi chi tiết: đáp án đúng, câu trả lời của từng tầng, độ tin cậy của *từng* ứng viên cộng hàng NULL.
- Nút tải toàn bộ bảng kết quả ra CSV (mã hoá `utf-8-sig` để mở đúng trong Excel tiếng Việt).

5.4 Đường dẫn REST

Table 8: Hai endpoint của `src/api.py` (đã kiểm chứng bằng `curl`)

Method	Đường dẫn	Nội dung
GET	<code>/health</code>	Trạng thái, thiết bị suy luận, số tham số encoder.
POST	<code>/answer</code>	Vào: <code>question</code> , <code>context</code> , <code>top_k</code> , <code>tau_null</code> . Ra: <code>answer</code> , <code>abstained</code> , <code>confidence</code> , danh sách ứng viên kèm <code>proba</code> và tọa độ ký tự, <code>null_proba</code> , số cửa sổ, phương án model một tầng sẽ chọn.

5.5 Ảnh chụp màn hình

5.6 Giới hạn của bản demo

- Ứng dụng đơn tiến trình, không có xác thực hay hàng đợi; tải đồng thời sẽ xếp hàng trước cùng một GPU.
- Trọng số 537 MB nằm ngoài phạm vi mã nguồn; người chạy phải đặt `models/phobert_qa/` và `models/reranker/reranker.pkl` đúng đường dẫn, nếu không app dừng ở màn hình báo lỗi nạp mô hình.
- File `app/visualization_helpers.py` (174 dòng, vẽ attention heatmap) là thành phần còn sót lại từ một phiên bản trước và **không được** `app.py` import (hoặc: sử dụng). Nó được giữ lại trong repo nhưng không phải một tính năng đang chạy.

6 Kết luận và hướng phát triển

6.1 Kết luận

Đồ án đã hoàn thành một hệ thống hỏi—đáp trích xuất tiếng Việt chạy được từ đầu đến cuối trên một GPU 4GB, với bốn đóng góp đo được:

- Pipeline tự viết hoàn toàn cho phần ánh xạ tọa độ — từ làm phẳng JSON sang Parquet, tự trải cửa sổ trượt 256/64, tự sinh `offset_mapping` — thay vì dùng `tokenizer(..., return_offsets=True)` của thư viện. Toàn bộ tham số huấn luyện được lưu và tái lập từ `training_args.bin` với `seed = 42`.
- Kết quả kiểm chứng lại được trên UIT-ViQuAD 2.0 test: $EM = 42.40$, $F_1 = 57.90$, HasAns $F_1 = 70.62$, NoAns = 31.78, và đặc biệt $F_1 = 79.16$ trên phần câu hỏi model chịu trả lời.
- Một hồ sơ lỗi định lượng, phân hoạch đủ 2882 câu thành tám nhóm loại trừ nhau, chỉ ra rằng hai khiếm khuyết lớn nhất là *cắt sai biên* (608 câu, 21.10%) và *không dám im lặng* (644 câu, 22.35%) — chứ không phải thiếu năng lực định vị thông tin.
- Tầng hai mang lại quyền từ chối có hiệu chuẩn: +21.21 điểm bắt câu bẫy trên mẫu so sánh, đổi lấy -7.12 điểm HasAns F_1 , và toàn bộ đánh đổi đó điều khiển được bằng một tham số τ duy nhất ngay trên giao diện.

6.2 Hướng phát triển

Chính hồ sơ lỗi chỉ ra bước tiếp theo, và nó nằm ở kiến trúc chứ không ở hậu xử lý:

- **Thay head pointer bằng span scorer:** chấm điểm trực tiếp từng cặp (i, j) thay vì cộng rồi hai logit. Đây là kết luận rút ra từ việc 92.8% đáp án đúng đã nằm sẵn trong pool nhưng reranker bề mặt chỉ sửa được 12/821 câu — nếu span scorer không thắng pointer trong một thử nghiệm nhỏ, hướng này nên bỏ.
- **Reranker dùng biểu diễn học được:** thay 34 đặc trưng bề mặt bằng vectơ câu hỏi + vectơ span lấy từ chính encoder. Chi phí là phải huấn luyện lại trên pool đã xuất, nên chỉ đáng làm nếu pilot với encoder đóng cho thấy span scorer thắng.
- **Đo hai tầng trên toàn bộ test:** hiện chỉ có trên mẫu 200 câu; một lượt full test sẽ đóng luôn khoảng trống “ Δ mang sai số lấy mẫu” đã nêu ở mục 4.10.
- **Triển khai:** trọng số 537 MB cần một nguồn phát hành riêng (Google Drive hoặc Hugging Face Hub) vì dung lượng này không phù hợp với kho lưu trữ mã nguồn.

7 References

- [1] K. V. Nguyen, D.-V. Nguyen, A. G.-T. Nguyen và N. L.-T. Nguyen, “UIT-ViQuAD: A manually reviewed Vietnamese dataset for extractive question answering”, *Proceedings of the 28th International Conference on Computational Linguistics (COLING 2020)*, tr. 2595–2605, 2020.
- [2] K. V. Nguyen, S. Q. Tran, L. T. Nguyen, T. V. Huynh, S. T. Luu và N. L.-T. Nguyen, “VLSP 2021 – ViMRC Challenge: Vietnamese Machine Reading Comprehension”, *The 8th International Workshop on Vietnamese Language and Speech Processing (VLSP 2021)*, arXiv:2203.11400, 2021.

-
- [3] P. Rajpurkar, R. Jia và P. Liang, “Know What You Don’t Know: Unanswerable Questions for SQuAD”, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018)*, tr. 1071–1076, 2018.
- [4] D. Q. Nguyen và A. T. Nguyen, “PhoBERT: Pre-trained language models for Vietnamese”, *Findings of the Association for Computational Linguistics: EMNLP 2020*, tr. 1012–1025, 2020.
- [5] Y. Liu và cộng sự, “RoBERTa: A Robustly Optimized BERT Pretraining Approach”, arXiv:1907.11692, 2019.
- [6] A. Conneau và cộng sự, “Unsupervised Cross-lingual Representation Learning at Scale”, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, tr. 8440–8451, 2020.
- [7] J. Devlin và cộng sự, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *Proceedings of NAACL-HLT 2019*, tr. 4171–4186, 2019.
- [8] I. Loshchilov và F. Hutter, “Decoupled Weight Decay Regularization”, *International Conference on Learning Representations (ICLR 2019)*, arXiv:1711.05101, 2019.
- [9] T. Chen, B. Xu, C. Zhang và C. Guestrin, “Training Deep Nets with Sublinear Memory Cost”, arXiv:1604.06174, 2016.
- [10] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine”, *The Annals of Statistics*, tập 29, số 5, tr. 1189–1232, 2001.
- [11] T. Wolf và cộng sự, “Transformers: State-of-the-Art Natural Language Processing”, *Proceedings of EMNLP 2020 – System Demonstrations*, tr. 38–45, 2020.
- [12] F. Pedregosa và cộng sự, “Scikit-learn: Machine Learning in Python”, *Journal of Machine Learning Research*, tập 12, tr. 2825–2830, 2011.
- [13] Streamlit Inc., “Streamlit Documents – caching guide”, <https://docs.streamlit.io/>, truy cập 2026.